

## 1. Introducción

El presente proyecto consiste en el desarrollo de un sistema experto utilizando el paradigma de programación lógica mediante Prolog, integrado con tecnologías modernas como Node.js para su exposición a través de una API REST.

Un sistema experto es un programa capaz de emular la toma de decisiones de un humano experto en un área específica, utilizando reglas y hechos definidos en una base de conocimiento. En este caso, el sistema se enfoca en la evaluación de contratos para determinar si se aplica una penalización, en función de ciertas condiciones previamente establecidas.

El objetivo principal del proyecto es demostrar la integración entre diferentes paradigmas de programación, así como el uso de Prolog para la resolución de problemas basados en lógica y reglas. Además, se busca crear una aplicación funcional que permita realizar consultas de manera sencilla mediante peticiones HTTP.

---

## 2. Paradigmas de programación utilizados

En el desarrollo de este proyecto se emplearon dos paradigmas principales:

### 2.1 Programación lógica

La programación lógica se implementó mediante el uso de Prolog (Tau Prolog). Este paradigma se basa en la definición de hechos y reglas, en lugar de instrucciones paso a paso.

En la base de conocimiento (knowledge.pl), se definen elementos como contratos y condiciones, y posteriormente se establecen reglas que permiten inferir nueva información. Por ejemplo:

- Definición de hechos (contratos existentes)
- Condiciones (pagos tardíos, incumplimientos, etc.)
- Reglas (determinación de penalizaciones)

El motor de inferencia de Prolog permite responder consultas automáticamente, sin necesidad de especificar cómo resolverlas paso a paso.

---

## 2.2 Programación imperativa

La programación imperativa se utilizó mediante Node.js y Express para crear el servidor que gestiona las solicitudes HTTP.

Este paradigma se basa en instrucciones secuenciales y control de flujo. En este caso, se utiliza para:

- Recibir peticiones del cliente
- Procesar las consultas
- Ejecutar el motor de Prolog
- Devolver resultados en formato JSON

La combinación de ambos paradigmas permite separar la lógica del negocio (Prolog) de la infraestructura (Node.js).

---

## 3. Arquitectura del sistema

El sistema sigue una arquitectura de tipo cliente-servidor, organizada de la siguiente manera:

### Componentes principales:

#### 1. Cliente

- Puede ser Thunder Client o cualquier aplicación que haga peticiones HTTP
- Envía consultas en formato JSON

#### 2. Servidor (Node.js + Express)

- Recibe las solicitudes en el endpoint /query
- Procesa la consulta enviada
- Ejecuta el motor de Prolog

#### 3. Motor lógico (Tau Prolog)

- Carga la base de conocimiento (knowledge.pl)
- Evalúa las consultas
- Genera resultados mediante inferencia lógica

#### 4. Base de conocimiento (Prolog)

- Contiene hechos y reglas
- Define la lógica del sistema experto

---

#### Flujo de funcionamiento:

1. El cliente envía una consulta (por ejemplo: `penalty_applicable(X).`)
  2. El servidor recibe la petición
  3. Se ejecuta la consulta en Tau Prolog
  4. Se obtienen los resultados
  5. El servidor responde en formato JSON
- 

#### 4.Ejemplo de ejecución

The screenshot shows a REST client interface with the following details:

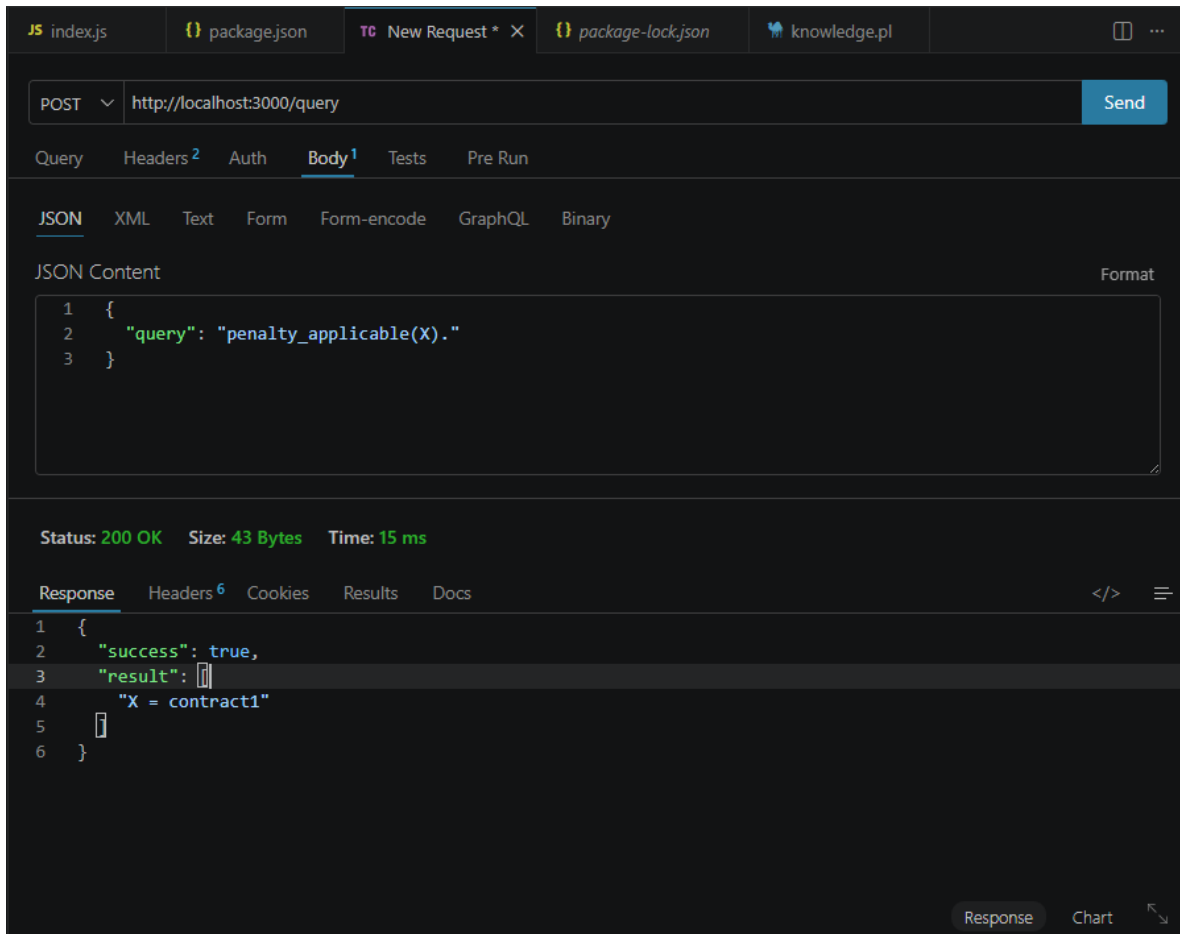
- Request:**
  - Method: POST
  - URL: `http://localhost:3000/query`
  - Body (JSON):

```
1 {
2   "query": "penalty_applicable(contract1)."
3 }
```
- Response:**
  - Status: 200 OK
  - Size: 34 Bytes
  - Time: 24 ms
  - Body (JSON):

```
1 {
2   "success": true,
3   "result": [
4     "true"
5   ]
6 }
```

### Interpretación:

Esto confirma que el contrato evaluado cumple con las reglas definidas en la base de conocimiento.



### Interpretación:

El resultado indica que existe al menos un contrato (contract1) que cumple con las condiciones necesarias para que se aplique una penalización.

---

## 5. Conclusiones y extensiones

El desarrollo de este proyecto permitió comprender la importancia de los sistemas expertos y su aplicación en la resolución de problemas basados en reglas.

El uso de Prolog facilita la representación del conocimiento de forma declarativa, permitiendo que el sistema realice inferencias automáticamente. Por otro lado,

Node.js proporciona una forma eficiente de exponer este conocimiento mediante servicios web.

La integración de ambos paradigmas demuestra que es posible combinar diferentes enfoques de programación para construir soluciones más robustas y flexibles.

---

**Posibles extensiones del proyecto:**

- Implementar una interfaz gráfica para mejorar la interacción con el usuario
- Agregar explicaciones de las inferencias realizadas por el sistema
- Ampliar la base de conocimiento con más reglas y casos
- Integrar una base de datos para manejar información dinámica
- Desplegar el sistema en la nube para acceso remoto